

FACULDADE ENGENHEIRO SALVADOR ARENA  
ENGENHARIA DA COMPUTAÇÃO  
ESTRUTURAS DE DADOS – PROJETO I  
1º SEMESTRE/2026  
PROF. ISRAEL FLORENTINO

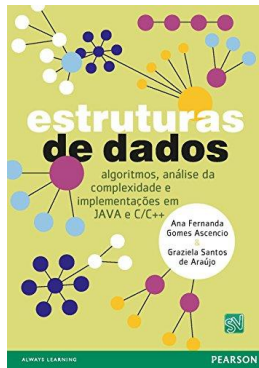
**PROJETO: BENCHMARK EXPERIMENTAL**

---



Nossos **objetivos** nesta aula são:

- compreender algoritmos de ordenação e busca.
- implementar algoritmos em C++.
- instrumentar métricas de execução.
- executar experimentos com dados variados.
- analisar estatisticamente os resultados.
- comparar resultados experimentais e teóricos.



Para esta aula, foram utilizados como referência o **Capítulo 2 (Algoritmos de Ordenação e Busca)** do nosso livro-texto:

ASCENCIO, Ana Fernanda Gomes; ARAÚJO, Graziela Santos de. Estruturas de Dados: algoritmos, análise da complexidade e implementações em JAVA e C/C++. **São Paulo: Perarson Prentice Halt**, v. 3, 2010.

*Não deixem de ler depois desta aula!*

---

**PROJETO ESTRUTURAS DE DADOS**

---

Benchmark Experimental e Análise Assintótica de Algoritmos de Ordenação e Busca em C++

**OBJETIVO GERAL**

---

Desenvolver um sistema em C++ capaz de implementar, instrumentar e analisar algoritmos clássicos de ordenação e busca, realizando medições empíricas, análise estatística e comparação com a análise assintótica formal.

## CONTEÚDOS ENVOLVIDOS

---

- Fundamentos de Estruturas de Dados
- Linguagem C/C++
- Análise Assintótica ( $O$ ,  $\Theta$ ,  $\Omega$ )
- Modelo RAM
- Algoritmos de Ordenação
- Algoritmos de Busca
- Recursão
- Análise Estatística Aplicada

## PARTE 1 – GERAÇÃO DE DADOS

---

- o sistema deve gerar vetores com os seguintes tamanhos (escala obrigatória)
  - 1.000, 5.000, 10.000, 50.000, 100.000, 500.000, 1.000.000
  - cenários obrigatórios (mínimo 6)
    - dados aleatórios (uniforme)
    - dados ordenados
    - dados em ordem reversa
    - dados parcialmente ordenados (ex.: 80% ordenado)
    - dados com alta repetição (ex.: valores entre 0 e 100)
    - dados com distribuição “quase real” (ex.: gaussiana simulada)
- reprodutibilidade obrigatória
- usar *seed fixa* configurável (ex.: 42) e registrar no relatório

## PARTE 2 – IMPLEMENTAÇÃO DOS ALGORITMOS

---

- algoritmos de ordenação obrigatórios
  - Bubble Sort
  - Insertion Sort
  - Selection Sort
  - Merge Sort
  - Quick Sort (pivô simples)
  - Quick Sort randomizado (obrigatório)
  - Heap Sort
  - Shell Sort
- algoritmos de ordenação opcionais
  - Counting Sort (quando aplicável)
  - Radix Sort (quando aplicável)
- algoritmos de busca obrigatórios
  - busca sequencial
  - busca binária
  - busca binária instrumentada (comparações + acessos)
- algoritmos de Busca obrigatórios
  - busca sequencial
  - busca binária
  - busca binária instrumentada (comparações + acessos)

## PARTE 3 – INSTRUMENTAÇÃO

---

- cada algoritmo deverá instrumentar e registrar, por execução:
  - número de comparações
  - número de trocas/movimentações
  - número de acessos ao vetor (leituras + escritas)
  - profundidade máxima de recursão (quando houver)
  - tempo de execução (std::chrono, em nanossegundos)
  - memória auxiliar estimada (bytes, por exemplo, Merge Sort)
- repetir cada experimento 30 execuções por configuração ( $n \times \text{cenário} \times \text{algoritmo}$ )
- calcular estatísticas a partir das 30 execuções (média e dispersão)

## PARTE 4 – ANÁLISE ESTATÍSTICA

---

- estatísticas obrigatórias (por algoritmo,  $n$  e cenário)
  - média
  - desvio padrão
  - mínimo e máximo
  - mediana
  - coeficiente de variação (CV)
  - intervalo de confiança 95% da média
- gráficos obrigatórios (mínimo 4)
  - $tempo \times n$  (escala linear)
  - $tempo \times n$  (escala log-log)
  - comparações  $\times n$
  - crescimento experimental vs crescimento teórico (normalizado)  
(ex.:  $tempo / (n \log n)$  e  $tempo / n^2$ )

## PARTE 5 – ANÁLISE ASSINTÓTICA FORMAL

---

- para cada algoritmo
  - apresentar a função teórica  $T(n)$  (pior caso e caso médio quando aplicável)
  - demonstrar formalmente  $O(g(n))$  com definição e constantes ( $c$  e  $n_0$ )
  - estimar experimentalmente o crescimento via ajuste (ex.: regressão em log-log)
  - comparar crescimento empírico com  $g(n)$  e justificar divergências
  - classificar usando Big O()

## PARTE 6 – RELATÓRIO TÉCNICO

---

- estrutura obrigatória do relatório (formato mini paper):
  - introdução e objetivos
  - fundamentação teórica (ordenação, busca, estabilidade, pior/médio caso)
  - modelo RAM e critérios de contagem
  - metodologia experimental (hardware/OS/compilador/flags, seed, repetições)
  - resultados (tabelas e gráficos)
  - análise estatística (IC95%, CV, mediana)
  - comparação teoria vs prática (inclui normalizações)
  - discussão crítica (limitações e ameaças à validade)
  - conclusão e trabalhos futuros

## DISCUSSÃO CRÍTICA OBRIGATÓRIA

---

- por que Bubble Sort é inviável para grandes volumes?
- em quais cenários Insertion Sort supera algoritmos  $O(n \log n)$ ?
- por que Quick Sort pode degradar para  $O(n^2)$  e como o randomizado reduz isso?
- Heap Sort é estável? como provar/mostrar com exemplo?
- busca binária sempre é superior? (considerar custo de ordenar + cache/localidade)
- o ambiente influencia o tempo real? (CPU, carga, otimização, flags)
- por que a análise assintótica não garante desempenho prático?
- como repetição de valores afeta Quick/Merge/Heap?
- por que bibliotecas usam algoritmos híbridos (ex.: introsort/timsort)?
- qual ameaça à validade mais impacta seu experimento e como mitigou?

## CRITÉRIOS DE AVALIAÇÃO

---

- implementação correta e modular (OO + organização) – 20%
- instrumentação avançada (tempo, comparações, acessos, recursão, memória) – 15%
- metodologia experimental e reprodutibilidade (seed, 30 execuções, ambiente) – 10%
- análise estatística completa (IC95%, CV, mediana, gráficos) – 20%
- análise assintótica formal (demonstração + confronto empírico) – 20%
- qualidade do relatório (clareza, tabelas, interpretação crítica) – 10%
- comparação com `std::sort` e `std::stable_sort` – 5%